

SQL Training Handbook for Beginners

A 10-Day Practical Guide Using company_db

Sample Database: company_db

About This Handbook

This is a trainer's notes + beginner handbook for freshers who have little or no SQL knowledge. By the end of Day 10, you should be able to write basic-to-intermediate SQL queries confidently and answer common beginner SQL interview questions.

We use one database throughout: `company_db` — so every concept builds on what you already know.

Remember: Read the concept, run the example yourself, do the practice exercises, then move to the next topic. Don't rush.

Our Sample Database: `company_db`

departments

department_id	department_name	location
1	IT	Hyderabad
2	HR	Mumbai
3	Finance	Pune
4	Sales	Bangalore

employees

employee_id	employee_name	email	department_id	salary	joining_date
101	Rahul	rahul@gmail.com	1	70000	2022-01-10
102	Priya	priya@gmail.com	1	85000	2021-06-15
103	Amit	amit@gmail.com	2	60000	2023-03-20
104	Sneha	sneha@gmail.com	3	90000	2020-08-11
105	Raj	raj@gmail.com	4	75000	2022-11-05

We will add a few more small tables (customers, orders, products) only in the Joins/Subqueries sections, when they help illustrate a concept.

We'll write examples using Oracle syntax as the primary style, and point out MySQL differences where they matter.

DAY 1 — SQL Basics

1. Introduction to Databases and SQL

What is data? Data is just raw facts — a name, a number, a date. On its own it doesn't tell you much.

What is a database? A database is an organized place to store related data so it can be stored, searched, and updated easily — instead of scattering it across files.

What is a DBMS? A Database Management System (DBMS) is the software that lets you create, read, update, and delete data in a database (e.g., Oracle, MySQL, SQL Server).

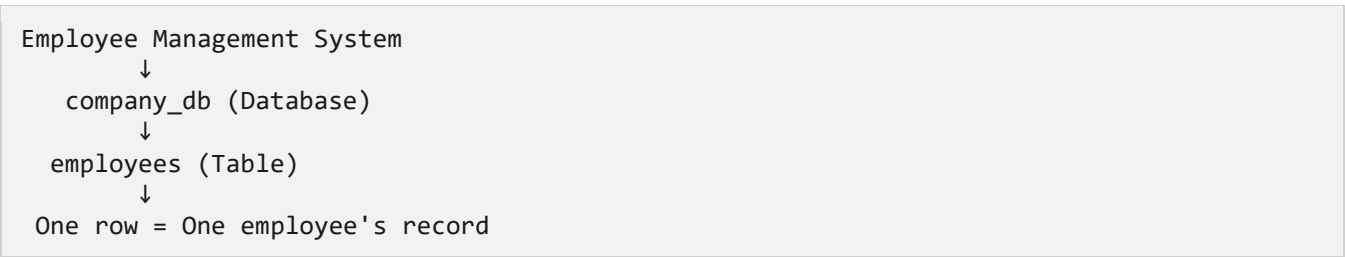
What is an RDBMS? A Relational DBMS stores data in tables (rows and columns) and lets tables relate to each other using keys. Oracle, MySQL, and PostgreSQL are all RDBMS.

What is SQL? SQL (Structured Query Language) is the language we use to talk to an RDBMS — to create tables, insert data, and ask questions ("queries") about that data.

Why do we need SQL? Because almost every real-world application (banking, e-commerce, HR systems) stores its data in a relational database, and SQL is the standard way to interact with it.

What is a table? A table is a structured grid of data — like a spreadsheet — with fixed columns and any number of rows.

Row vs Column: A column defines what kind of data is stored (e.g., employee_name, salary) — fixed when the table is created. A row is one record — one employee, one order, one product.



2. Relational vs Non-Relational Databases

Relational (SQL)	Non-Relational (NoSQL)
Data stored in tables (rows/columns)	Data stored as documents, key-value pairs, etc.
Fixed schema (structure)	Flexible / dynamic schema
Uses SQL	Uses database-specific query methods
Examples: MySQL, Oracle, PostgreSQL	Examples: MongoDB, Redis
Good for structured, relational data	Good for unstructured or fast-changing data

This handbook focuses only on relational databases (SQL). We won't go deeper into NoSQL.

3. SQL Environment Setup

To practice, you need a database server and a client tool:

- Server: the actual database engine running (e.g., MySQL Server, Oracle Database) — this is where your data lives.
- MySQL Workbench (or Oracle SQL Developer): a visual tool to write and run SQL queries against the server.

Term	Meaning
Host	Address of the machine running the server (often localhost)
Port	Network port the server listens on (MySQL default: 3306)
Username	Your login name for the database
Password	Your login password
Database	The specific database you want to work in (e.g., company_db)

Remember: Install MySQL Community Server + MySQL Workbench to follow along with this handbook practically.

DAY 2–3 — SQL Fundamentals

4. SQL Data Types

Type	Used For
INT	Whole numbers (e.g., employee_id)
BIGINT	Very large whole numbers
DECIMAL	Exact decimal numbers (e.g., money)
VARCHAR	Variable-length text
CHAR	Fixed-length text
TEXT	Long text (paragraphs, descriptions)
DATE	Date only (2024-05-10)
DATETIME	Date + time
TIMESTAMP	Date + time, often auto-updated
BOOLEAN	True/False values

CHAR vs VARCHAR: CHAR(10) always reserves 10 characters, padding with spaces. VARCHAR(10) only uses as much space as needed, up to 10 characters. Use VARCHAR for most text like names and emails.

INT vs BIGINT: INT handles typical whole numbers. Use BIGINT only when numbers might exceed ~2.1 billion (rare for beginner projects).

DECIMAL vs FLOAT: DECIMAL stores exact values — always use it for money (salary, price). FLOAT is approximate and can introduce rounding errors, so avoid it for financial data.

Common Mistake: Using FLOAT for salary/price columns. Always use DECIMAL(10,2) for money.

5. Creating and Selecting Databases

```
CREATE DATABASE company_db;

USE company_db;
```

CREATE DATABASE makes a new, empty database. USE tells the server "all my next commands apply to this database."

To remove a database entirely (deletes everything inside it):

```
DROP DATABASE company_db;
```

Common Mistake: DROP DATABASE is permanent — there's no undo. Never run it casually on a real project.

6. Creating Tables

```
CREATE TABLE employees (  
  employee_id INT PRIMARY KEY,  
  employee_name VARCHAR(100),  
  email VARCHAR(100),  
  department_id INT,  
  salary DECIMAL(10,2),  
  joining_date DATE  
);
```

- employee_id INT PRIMARY KEY — a whole number that uniquely identifies each employee.
- employee_name VARCHAR(100) — text, up to 100 characters.
- email VARCHAR(100) — text for the email address.
- department_id INT — links to the departments table.
- salary DECIMAL(10,2) — a number with up to 10 digits total, 2 after the decimal point.
- joining_date DATE — the date the employee joined.

Practice

1. Write a CREATE TABLE statement for departments with department_id, department_name, and location.
2. Add a customers table with customer_id, customer_name, and city.

7. Constraints

Constraint	What It Means	Why We Need It
PRIMARY KEY	Uniquely identifies each row, cannot be NULL	Prevents duplicate/blank identifiers
FOREIGN KEY	Links a column to another table's primary key	Keeps related data consistent
UNIQUE	No two rows can have the same value	e.g., ensures no duplicate emails
CHECK	Restricts values allowed in a column	e.g., salary must be > 0
DEFAULT	Provides an automatic value if none given	e.g., default status = 'Active'

PRIMARY KEY example:

```
employee_id INT PRIMARY KEY
```

FOREIGN KEY example (linking employees to departments):

```
department_id INT,  
FOREIGN KEY (department_id) REFERENCES departments(department_id)
```

UNIQUE example:

```
email VARCHAR(100) UNIQUE
```

CHECK example:

```
salary DECIMAL(10,2) CHECK (salary > 0)
```

DEFAULT example:

```
status VARCHAR(20) DEFAULT 'Active'
```

8. SQL Command Types

Category	Stands For	Commands	Purpose
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE	Define/change table structure
DML	Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Work with the data itself
DCL	Data Control Language	GRANT, REVOKE	Control user permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manage transactions

DELETE vs TRUNCATE vs DROP (Important!)

Command	What It Does	Can You Rollback?	Structure Removed?
DELETE	Removes specific rows (with WHERE)	Yes	No — table stays
TRUNCATE	Removes all rows, fast	Usually No	No — table stays
DROP	Removes the entire table structure	No	Yes — table is gone

Remember: DELETE = remove some/all rows but keep the table. TRUNCATE = empty the table quickly. DROP = delete the table itself, including its structure.

9. INSERT

Single row, all columns (in order):

```
INSERT INTO employees  
VALUES (106, 'Vikas', 'vikas@gmail.com', 1, 65000, '2023-07-01');
```

Specific columns only:

```
INSERT INTO employees (employee_id, employee_name, salary)  
VALUES (106, 'Vikas', 65000);
```

Any column not listed will get NULL or its DEFAULT value.

Multiple rows at once:

```
INSERT INTO employees (employee_id, employee_name, department_id, salary, joining_date)  
VALUES  
(107, 'Anita', 2, 62000, '2023-01-15'),
```

```
(108, 'Kiran', 4, 71000, '2022-09-09');
```

Practice

3. Insert yourself as a new employee with a made-up salary and department.
4. Insert two new rows into departments in a single statement.

10. UPDATE

```
UPDATE employees  
SET salary = 75000  
WHERE employee_id = 101;
```

This changes Rahul's salary to 75000.

Why WHERE matters:

If you forget the WHERE clause:

```
UPDATE employees  
SET salary = 75000;
```

This updates every single row in the table — all employees get salary 75000! Always double-check your WHERE clause before running an UPDATE.

Common Mistake: Running UPDATE without WHERE on a live table. Always SELECT with the same WHERE first to preview which rows will be affected.

11. DELETE

```
DELETE FROM employees  
WHERE employee_id = 106;
```

This removes only Vikas's row. Just like UPDATE, forgetting WHERE here deletes all rows in the table.

Practice

5. Update Amit's salary to 65000.
6. Delete the employee with employee_id = 108.

DAY 4 — Querying Data

12. SELECT

```
SELECT * FROM employees;
```

* means "all columns." Good for quick checks, but not ideal in real applications.

```
SELECT employee_name, salary  
FROM employees;
```

Why avoid SELECT *? It pulls unnecessary columns, wastes memory/network, and can break your application if the table structure changes later. Always select only the columns you actually need.

13. DISTINCT

```
SELECT DISTINCT department_id  
FROM employees;
```

Returns each unique department_id only once, removing duplicates.

14. WHERE

```
SELECT * FROM employees  
WHERE department_id = 1;
```

Filters rows so only matching ones are returned — here, only IT department employees.

15. Comparison Operators

Operator	Meaning
=	Equal to
<>	Not equal to
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal

```
SELECT * FROM employees  
WHERE salary >= 70000;
```

16. AND / OR / NOT

```
SELECT * FROM employees
WHERE department_id = 1 AND salary > 70000;
```

Both conditions must be true.

```
SELECT * FROM employees
WHERE department_id = 1 OR department_id = 4;
```

Either condition can be true.

```
SELECT * FROM employees
WHERE NOT department_id = 1;
```

Excludes IT department employees.

17. BETWEEN

```
SELECT *
FROM employees
WHERE salary BETWEEN 60000 AND 80000;
```

Includes both 60000 and 80000 (inclusive range).

18. IN

```
SELECT *
FROM employees
WHERE department_id IN (1, 2, 3);
```

Same as department_id = 1 OR department_id = 2 OR department_id = 3, but shorter and cleaner to read.

19. LIKE

% matches any number of characters, _ matches exactly one character.

```
-- Starts with 'R'
SELECT * FROM employees WHERE employee_name LIKE 'R%';

-- Ends with 'a'
SELECT * FROM employees WHERE employee_name LIKE '%a';

-- Contains 'am'
SELECT * FROM employees WHERE employee_name LIKE '%am%';
```

20. IS NULL

NULL means "unknown" or "no value" — it is not the same as zero or an empty string.

```
-- WRONG – this will never match, even if manager_id has no value
WHERE manager_id = NULL;

-- CORRECT
WHERE manager_id IS NULL;
```

Remember: You can never compare to NULL using =. Always use IS NULL or IS NOT NULL.

21. ORDER BY

```
SELECT * FROM employees
ORDER BY salary DESC;
```

ASC = ascending (default), DESC = descending.

```
SELECT * FROM employees
ORDER BY department_id ASC, salary DESC;
```

Sorts by department first, then by salary (highest first) within each department.

22. LIMIT

MySQL syntax:

```
SELECT * FROM employees
LIMIT 3;
```

Returns only the first 3 rows.

Remember: Oracle uses different syntax for limiting rows, such as FETCH FIRST 3 ROWS ONLY. We won't go deep into Oracle's row-limiting here.

23. Aliases

Column alias:

```
SELECT employee_name AS name
FROM employees;
```

Table alias:

```
SELECT e.employee_name
FROM employees e;
```

Aliases make queries shorter and more readable — especially useful once we start joining tables.

Practice

7. List all employees sorted by salary, highest first.
8. Find employees whose name starts with 'S' and earn more than 60000.
9. Find employees with no email recorded (assume some rows have NULL emails).

DAY 5 — SQL Functions

24. Aggregate Functions

Function	Purpose
COUNT	Number of rows
SUM	Total of a numeric column
AVG	Average value
MIN	Smallest value
MAX	Largest value

```
SELECT COUNT(*) FROM employees;           -- total number of employees
SELECT SUM(salary) FROM employees;        -- total salary paid
SELECT AVG(salary) FROM employees;        -- average salary
SELECT MIN(salary), MAX(salary) FROM employees;
```

COUNT(*) vs COUNT(column): COUNT(*) counts all rows, including ones with NULL values. COUNT(email) counts only rows where email is not NULL.

25. String Functions

```
SELECT CONCAT(employee_name, ' - ', email) FROM employees; -- joins text together
SELECT LENGTH(employee_name) FROM employees;              -- number of characters
SELECT UPPER(employee_name) FROM employees;               -- uppercase
SELECT LOWER(employee_name) FROM employees;               -- lowercase
SELECT SUBSTRING(employee_name, 1, 3) FROM employees;     -- first 3 characters
SELECT REPLACE(email, 'gmail.com', 'company.com') FROM employees; -- replace text
```

26. Date Functions

```
SELECT CURRENT_DATE; -- today's date
SELECT EXTRACT(YEAR FROM joining_date) FROM employees; -- pull out the year
```

Oracle-specific examples:

```
SELECT ADD_MONTHS(joining_date, 6) FROM employees; -- add 6 months
SELECT MONTHS_BETWEEN(SYSDATE, joining_date) FROM employees; -- months since joining
```

Remember: ADD_MONTHS and MONTHS_BETWEEN are Oracle-specific. In MySQL, similar results come from DATE_ADD(joining_date, INTERVAL 6 MONTH) and TIMESTAMPDIFF(MONTH, joining_date, CURDATE()).

27. Mathematical Functions

```

SELECT ROUND(salary/12, 2) FROM employees;  -- round to 2 decimal places (monthly salary)
SELECT CEIL(4.2) FROM dual;                -- rounds up -> 5
SELECT FLOOR(4.8) FROM dual;               -- rounds down -> 4
SELECT ABS(-25) FROM dual;                 -- absolute value -> 25
SELECT MOD(10, 3) FROM dual;               -- remainder -> 1

```

28. Conditional Functions

CASE — like an if/else for query results:

```

SELECT employee_name, salary,
       CASE
         WHEN salary >= 80000 THEN 'HIGH'
         WHEN salary >= 60000 THEN 'MEDIUM'
         ELSE 'LOW'
       END AS salary_band
FROM employees;

```

COALESCE — returns the first non-NULL value in a list:

```

SELECT COALESCE(email, 'Not Provided') FROM employees;

```

If email is NULL, it shows 'Not Provided' instead.

NULLIF — returns NULL if two values are equal, otherwise returns the first value:

```

SELECT NULLIF(department_id, 1) FROM employees;

```

Returns NULL for IT employees (department_id = 1), and the actual value otherwise.

Practice

10. Show each employee's name in uppercase along with their salary rounded to the nearest hundred.
11. Classify employees into HIGH/MEDIUM/LOW salary bands using CASE.

DAY 6 — Grouping Data

29. GROUP BY

Why do we need it? Aggregate functions like SUM or AVG normally summarize the whole table. GROUP BY lets us get a summary per category — e.g., per department, instead of one number for everyone.

```
SELECT department_id, COUNT(*)
FROM employees
GROUP BY department_id;
```

Shows how many employees are in each department.

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id;
```

Shows the average salary within each department.

30. HAVING

```
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 70000;
```

Only shows departments whose average salary exceeds 70000 — filtering happens after grouping.

31. WHERE vs HAVING (Important!)

WHERE	HAVING
Filters individual rows	Filters groups (after GROUP BY)
Runs before grouping	Runs after grouping
Cannot use aggregate functions	Commonly used with aggregate functions

```
-- WHERE: filter rows before grouping
SELECT department_id, AVG(salary)
FROM employees
WHERE salary > 50000
GROUP BY department_id;

-- HAVING: filter groups after grouping
SELECT department_id, AVG(salary)
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 70000;
```

Practice

- 12.Count how many employees joined each year.
- 13.Find departments with more than 1 employee.

DAY 7 — Joins

32. Why Do We Need Joins?

Employee data lives in employees, but department names live in departments. If we just query employees, we only see department_id (a number), not the department name. JOIN lets us combine rows from two (or more) tables based on a matching column, so we can see meaningful, combined results.

33. INNER JOIN

```
SELECT e.employee_name, d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

- FROM employees e — start with the employees table, aliased e.
- INNER JOIN departments d — join with the departments table, aliased d.
- ON e.department_id = d.department_id — match rows where department IDs are equal.

Output:

employee_name	department_name
Rahul	IT
Priya	IT
Amit	HR
Sneha	Finance
Raj	Sales

INNER JOIN only returns rows that have a match in both tables.

34. LEFT JOIN

```
SELECT e.employee_name, d.department_name
FROM employees e
LEFT JOIN departments d
    ON e.department_id = d.department_id;
```

Remember: LEFT JOIN returns all records from the left table (employees) plus matching records from the right table (departments). If there's no match, department columns show NULL.

35. RIGHT JOIN

```
SELECT e.employee_name, d.department_name
FROM employees e
```

```
RIGHT JOIN departments d
  ON e.department_id = d.department_id;
```

Returns all records from the right table (departments) plus matches from the left. In practice, most developers prefer LEFT JOIN because it's easier to read — you can simply swap table order instead of switching keywords.

36. FULL OUTER JOIN

Conceptually, FULL OUTER JOIN returns all rows from both tables, matched where possible, with NULLs where there's no match on either side.

Remember: MySQL does not directly support FULL OUTER JOIN — alternative approaches (like combining LEFT JOIN and RIGHT JOIN with UNION) are needed. We won't go deep into that here.

37. CROSS JOIN

```
SELECT e.employee_name, d.department_name
FROM employees e
CROSS JOIN departments d;
```

This produces a Cartesian product — every employee paired with every department (5 employees × 4 departments = 20 rows). Rarely used directly, but important to understand.

38. SELF JOIN

A table can be joined with itself — useful for hierarchical data like employee-manager relationships.

```
SELECT e.employee_name AS employee, m.employee_name AS manager
FROM employees e
LEFT JOIN employees m
  ON e.manager_id = m.employee_id;
```

Here, employees is used twice (as e and m) to compare each employee to their manager, who is also a row in the same table.

39. USING vs ON

ON lets you compare columns with different names:

```
... ON e.department_id = d.department_id
```

USING is a shortcut when both tables have a column with the exact same name:

```
... JOIN departments USING (department_id)
```

40. Multiple Table JOIN

Example joining employees, departments, and a simple orders table (assume each order is handled by one employee):

```
SELECT e.employee_name, d.department_name, o.order_amount
FROM employees e
```

```
INNER JOIN departments d ON e.department_id = d.department_id  
INNER JOIN orders o ON e.employee_id = o.employee_id;
```

Practice

14. List every employee with their department name using INNER JOIN.
15. List every department with its employees using LEFT JOIN, including departments with zero employees.
16. Write a SELF JOIN to show each employee alongside their manager's name.

DAY 8 — Subqueries

41. What is a Subquery?

A subquery is a query inside another query — the inner query runs first, and its result is used by the outer query.

```
SELECT employee_name
FROM employees
WHERE department_id = (
    SELECT department_id FROM departments WHERE department_name = 'IT'
);
```

42. Subquery in WHERE

Find employees earning more than the average salary:

```
SELECT employee_name, salary
FROM employees
WHERE salary > (
    SELECT AVG(salary) FROM employees
);
```

Step-by-step: First, the inner query `SELECT AVG(salary) FROM employees` calculates one number — the average salary. Then the outer query compares every employee's salary against that number.

43. Subquery in SELECT

```
SELECT employee_name, salary,
    (SELECT AVG(salary) FROM employees) AS company_avg_salary
FROM employees;
```

Shows each employee's salary next to the overall company average, for easy comparison.

44. Subquery in FROM

```
SELECT department_id, avg_salary
FROM (
    SELECT department_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY department_id
) dept_avg
WHERE avg_salary > 65000;
```

The inner query builds a temporary result set (department averages), and the outer query filters that result — useful when you need to filter on an already-aggregated value without using `HAVING`.

45. EXISTS / NOT EXISTS

```
SELECT department_name
FROM departments d
WHERE EXISTS (
    SELECT 1 FROM employees e WHERE e.department_id = d.department_id
)
```

```
);
```

Returns departments that have at least one employee. EXISTS checks only whether the subquery returns any rows — it doesn't care about the actual values.

46. ANY / ALL

```
-- Salary greater than ANY IT employee's salary (i.e., greater than the minimum)
SELECT employee_name FROM employees
WHERE salary > ANY (SELECT salary FROM employees WHERE department_id = 1);

-- Salary greater than ALL IT employees' salaries (i.e., greater than the maximum)
SELECT employee_name FROM employees
WHERE salary > ALL (SELECT salary FROM employees WHERE department_id = 1);
```

47. Set Operations

```
SELECT
employee_name
FROM
employees
UNION
SELECT
customer_name
FROM
customers;
```

Operation	Behavior
UNION	Combines results, removes duplicates
UNION ALL	Combines results, keeps duplicates (faster)
INTERSECT	Returns only rows common to both queries
MINUS / EXCEPT	Returns rows in the first query but not the second

Remember: Use UNION ALL when you know there won't be duplicates or don't care about them — it's faster since it skips the duplicate-removal step. Oracle uses MINUS; most other databases (including MySQL 8+) use EXCEPT.

Practice

- Find employees who earn more than the average salary of their own department (hint: correlated subquery).
- Find departments that currently have no employees using NOT EXISTS.

DAY 9 — Advanced SQL Basics (Awareness Level)

48. Indexes

What is an index? A database index is like the index at the back of a book — instead of scanning every page (row) to find something, the database jumps straight to it.

Why do we need it? Indexes make SELECT queries with WHERE, JOIN, or ORDER BY much faster on large tables.

```
CREATE INDEX idx_employee_department
ON employees(department_id);

DROP INDEX idx_employee_department ON employees; -- MySQL syntax
```

When is it useful? On columns frequently used in WHERE clauses or JOIN conditions, especially in large tables.

Basic disadvantages: Indexes speed up reads but slow down INSERT/UPDATE/DELETE slightly (since the index must also be updated), and they use extra storage.

49. Views

What is a View? A saved, reusable query that behaves like a virtual table.

Why use it? Simplifies complex queries, hides sensitive columns, and provides a consistent way to access commonly-needed data.

```
CREATE VIEW employee_details AS
SELECT employee_name, salary
FROM employees;

-- Use it just like a table:
SELECT * FROM employee_details;

DROP VIEW employee_details;
```

View vs Table: A table physically stores data. A view does not store data itself — it just stores a query definition, and runs that query fresh each time you use it.

50. Triggers

What is a trigger? A block of code that automatically runs when a specific event (INSERT, UPDATE, DELETE) happens on a table.

When does it execute? BEFORE or AFTER the triggering event.

Simple audit example (logs every salary change):

```
CREATE TRIGGER trg_salary_audit
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
```

```
INSERT INTO salary_audit_log (employee_id, old_salary, new_salary, changed_on)
VALUES (OLD.employee_id, OLD.salary, NEW.salary, NOW());
END;
```

51. Stored Procedures

What is it? A saved block of SQL code (with optional parameters) that you can call repeatedly, instead of retyping the same logic.

Why use it? Reusability, and moving repeated business logic into the database.

```
CREATE PROCEDURE GetEmployeesByDept(IN dept_id INT)
BEGIN
    SELECT * FROM employees WHERE department_id = dept_id;
END;

-- Call it:
CALL GetEmployeesByDept(1);
```

52. Functions

What is a database function? Similar to a procedure, but it always returns a single value and can be used directly inside a SELECT statement.

Procedure	Function
Performs an operation	Usually returns a value
Can have OUT parameters	Returns a value via RETURN
Called with CALL	Can often be used inside expressions/SELECT

```
CREATE FUNCTION GetAnnualSalary(monthly_salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN monthly_salary * 12;
END;

SELECT GetAnnualSalary(salary) FROM employees;
```

Remember: Exact syntax for procedures/functions/triggers varies between Oracle, MySQL, and other databases — the concepts above stay the same everywhere.

DAY 10 — Revision + Practice

53. SQL Revision — Quick Recap

- Basics: databases, DBMS, RDBMS, tables, rows/columns
- DDL: CREATE, ALTER, DROP, TRUNCATE
- DML: INSERT, SELECT, UPDATE, DELETE
- Filtering: WHERE, AND/OR/NOT, BETWEEN, IN, LIKE, IS NULL
- Sorting/Limiting: ORDER BY, LIMIT
- Functions: aggregate, string, date, math, conditional (CASE, COALESCE)
- Grouping: GROUP BY, HAVING, WHERE vs HAVING
- Joins: INNER, LEFT, RIGHT, FULL OUTER, CROSS, SELF
- Subqueries: in WHERE, SELECT, FROM; EXISTS, ANY/ALL; UNION/UNION ALL
- Advanced basics: indexes, views, triggers, procedures, functions

Mini Practice Set (Basic Interview-Style Problems)

19. Write a query to find the second highest salary in employees.
20. Find the department with the highest average salary.
21. List employees who do not belong to any department (assume some rows have NULL department_id).
22. Find employees who joined in the last 2 years.
23. Write a query to find duplicate email addresses, if any, in employees.
24. Get the count of employees per department, including departments with zero employees.
25. Find the employee with the highest salary in each department.

Final SQL Cheat Sheet

Commands

CREATE, ALTER, DROP, TRUNCATE
INSERT, SELECT, UPDATE, DELETE
COMMIT, ROLLBACK
GRANT, REVOKE

Filtering

WHERE, AND, OR, NOT
IN, BETWEEN, LIKE, IS NULL

Sorting / Grouping

ORDER BY, GROUP BY, HAVING
DISTINCT, LIMIT

Joins

INNER JOIN, LEFT JOIN, RIGHT JOIN
FULL OUTER JOIN, CROSS JOIN, SELF JOIN

Functions

COUNT, SUM, AVG, MIN, MAX
CONCAT, UPPER, LOWER, SUBSTRING
ROUND, CASE, COALESCE

Quick Reference: WHERE vs HAVING

WHERE -> filters rows, runs BEFORE GROUP BY
HAVING -> filters groups, runs AFTER GROUP BY

Quick Reference: DELETE vs TRUNCATE vs DROP

DELETE -> removes some/all rows, table stays, can rollback
TRUNCATE -> removes all rows fast, table stays, usually can't rollback
DROP -> removes the entire table structure

*End of Handbook — you're ready to write confident, practical SQL and tackle basic SQL interview questions.
Keep practicing on company_db!*